# A2L-RAGAlign: Calibrated Restoration of Embedded Measurement Interfaces from Firmware Symbols

Wei Wang
East China Normal University
Shanghai 200062, China
71285902039@stu.ecnu.edu.cn

Dehui Du
East China Normal University
Shanghai 200062, China
dhdu@sei.ecnu.edu.cn
Corresponding author

***Abstract*—Embedded development of a safety-relevant automotive control unit couples the firmware to its calibration toolchain through an ASAM MCD-2MC A2L description file—an executable contract binding every control variable to its address, raw bit layout, and physical scaling. Every recompilation invalidates part of that contract, and both restoration mechanisms—exact string matching and hand-maintained spreadsheets—fail on ordinary renames. We present A2L-RAGAlign, a retrieval-augmented framework that fuses retrieval and token evidence over an embedded knowledge base into calibrated alignments, and gates each with a structural contract check: the I/O class and type family a symbol name carries are compared with the candidate, and a class or type-family flip—a category error no score can rank out—is demoted by a penalty and a hard tier cap, so it can never be auto-accepted. On a held-out split of 314 renamed queries (from 631 generated over 83 signals) it reaches 99.0% Top-1 where exact matching scores 0% and string similarity 92.4%, and 97.6% on the 84 multi-hop renames; on 127 contract-violating pairs it removes false auto-acceptance entirely at 93.6% auto-coverage and 99.3% precision. On the production image itself (4,003 real symbols) it reaches 97.8% Top-1, and removing the retrieval channel costs 25.6 Top-1 points.**

***Index Terms*—A2L, calibration and debug toolchain, contract verification, embedded measurement interface, firmware evolution, retrieval-augmented alignment**

## I. Introduction

An automotive control unit (ECU) is developed, debugged, and calibrated through a hardware–software toolchain. The target firmware exposes its internal state over a measurement protocol (XCP [1]), and a calibration master (INCA, CANape) reads and writes it through a description file (MCD-2MC [2]) stating, for every object, where the variable lives, how many bits it occupies, which bit range is meaningful, and how raw counts become physical units. We call this the *embedded measurement interface*: not documentation but an executable contract—every bench number is the product of the address, layout, mask, and conversion it declares.

Firmware evolution breaks that contract continuously. Each recompilation moves symbols, renames identifiers, changes widths and bit layouts, and adds or removes variables, so the description must be resynchronized with the binary; the tool then reads the wrong word, bit range, or class of variable, and the failure is silent by construction—the master still connects, still plots, and the engineer sees a plausible value. Restoration today rests on *exact string matching* and *manually curated spreadsheets*; both break on ordinary divergence—toolchain-specific spellings, identifier styles (snake-case *vs.* camel-case, abbreviations *vs.* full words [3], [4]), struct-path qualification—so engineers fall back to serial lookup.

Restoring the contract is therefore not a naming problem but a *typed* one, with the shape of a compiler type check: two objects can agree on every character of their names and still describe different quantities. The missing ingredient is *domain knowledge retrieval*: the knowledge to repair a broken match already exists in an engineering organization—layout rules, naming conventions, calibration sheets, and the semantic descriptions engineers attach to signals. A2L-RAGAlign packages it as a retrieval-augmented framework layered on an operational update tool (Fig. 1), whose tiered automation delivers the trustworthiness safety-relevant development demands [5].

Our contributions are twofold.

- **Contract-level formalization.** Alignment as evidence fusion over an embedded knowledge base, with a per-field *contract check* on the two structural fields a name carries (I/O class, type family) that hard-caps the two category-level violations: class flip and type-family flip (Sec. III).
- **Calibrated, auditable automation.** Alignments are tiered AUTO/SUGGEST/MANUAL with thresholds calibrated on held-out signals, and every decision ships with its evidence in an alignment report, the configuration trail the description needs (Sec. III-C). Ranking is near saturation; our contribution is the gate.

## II. Related Work

### A. Embedded Measurement and Configuration Toolchain

The standards define our path: XCP [1] the transport, MCD-2MC [2] the description, MDF [6] the master data, ISO 14229 [7] diagnostic services over the same bus (CAN [8]); AUTOSAR [9] standardizes the software interfaces, and ELF/DWARF [10] is the type truth we bind to. Existing A2L tooling regenerates the address-bearing part

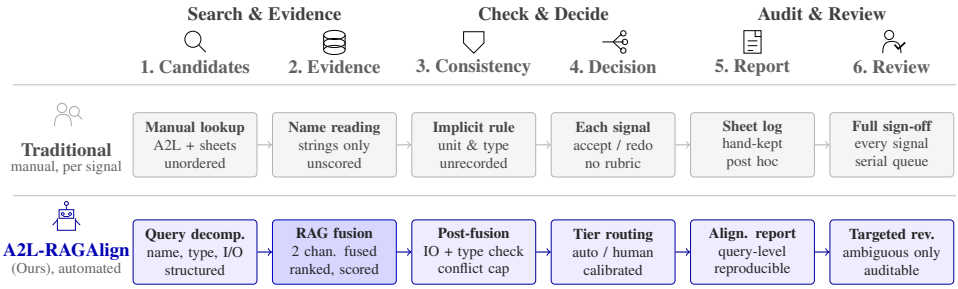



Fig. 1. The symbol-alignment workflow, manual versus tool-assisted. The manual route is per signal and unrecorded; A2L-RAGAlign produces a scored candidate, a consistency verdict, and a calibrated decision per query.

of a description from the image, while identity and physical metadata remain hand-carried—costly for safety-relevant software [5]. Automatic schema matching [11] addresses the metadata half—the abstract twin of our problem—but lacks the structural evidence generic matchers miss; our baselines follow [12].

### B. Retrieval-Augmented Software Engineering

Surveys chart the rise of LLM-based SE tooling [13], [14]; retrieval-augmented generation grounds model outputs in external knowledge [15], [16], with dense retrieval [17] and code search [18] as substrate; A2L-RAGAlign applies the RAG pattern to a configuration artifact, not source code, and stops short of generation: no language model is called, so scoring is deterministic and every decision stays auditable.

## III. Methodology

### A. Problem Setting: The Measurement Contract and Its Drift

**What the contract contains.** The A2L description is the executable contract of the ECU: its `MODULE` holds `MEASUREMENT` and `CHARACTERISTIC` objects, each referencing a data type, record layout, conversion method, and bit mask, with XCP segment definitions tying the description to the transport [1]. The base tool derives all of it from the firmware's ELF/DWARF debug information [10], expanding struct members, arrays, and bitfield masks.

**Three kinds of drift, and what they cost.** Identity drift renames variables; schema drift changes type width, bit layout, or class; address drift moves them at every recompilation. Table I lists the resulting error classes. Two are categorical: a name carries *identical* evidence for a class- or type-flipped twin, so no score can separate them—and a score that cannot must not auto-accept them.

**Scale.** The problem is not hypothetical: on a production 32-bit ARM transmission-control image, DWARF extraction yields 31,316 signals, of which exact matching resolves 28,812; the remaining 2,504 (**8.00 %**) stay unresolved and are silently dropped—variables that exist in the firmware but are invisible to the bench. We forensically separated that gap instead of assuming it: among the 580 distinct names, none survives exact or case-insensitive lookup in the address file, none appears in the generated description, and only 9 have even a looser counterpart, so 98.4 % are variables the address file does not export. The 8.00 % is a *symbol-coverage* gap no naming method can close; the naming divergence alignment *can* absorb is measured on the same image in Sec. IV-E.

TABLE I
Error taxonomy of a drifted measurement contract, with the measured detection rate of each class and the layer that owns it (Check = alignment contract check, Consist. = incremental consistency check).

| Drift class | At the target | Consequence | Owner |
|---|---|---|---|
| Class flip | exposed as MEAS | master writes const. | Check (100.0%) |
| Type-family | layout no longer fits | decodes to another | Check (100.0%) |
| Bit-layout | mask stops isolating | contaminated $2^w - 2^b$ | Consist. (100.0%) |
| Scaling loss | raw count, wrong $k$ | value off by $6\times$ | Consist. (100.0%) |
| Duplicate | two objects, one var | master reads stale | Consist. (100.0%) |
| Orphan | object vanishes | stale entry kept | Consist. (100.0%) |

### B. Knowledge Base and Decomposition

The knowledge base stores one document per known signal (name, description, unit, I/O class, C type) from three sources: **K1** rules—C-type to A2L type, record-layout, and conversion mappings plus naming-convention classifiers; **K2** historical configurations—per-signal annotations from calibration sheets; **K3** semantic corpora—names paired with physical-semantic descriptions. All ship with the artifact, and K2 is authoritative where they disagree. Each identifier is decomposed into a struct path, a stem, word tokens (snake- and camel-case boundaries), a type suffix (`u8`..`f64`, `b`), and an I/O suffix (`C` $\rightarrow$ calibration, `M` $\rightarrow$ measurement)—recovering the contract fields of Table I from the name itself.

### C. Confidence-Ranked Alignment, Completion, and Consistency

Given a query symbol, a retriever fetches the top-$k$ knowledge-base documents ($k$=5) through a pluggable protocol: a zero-dependency character $n$-gram cosine retriever (default and experimental baseline), an optional sentence-transformer backend [19], and optional FAISS or Chroma

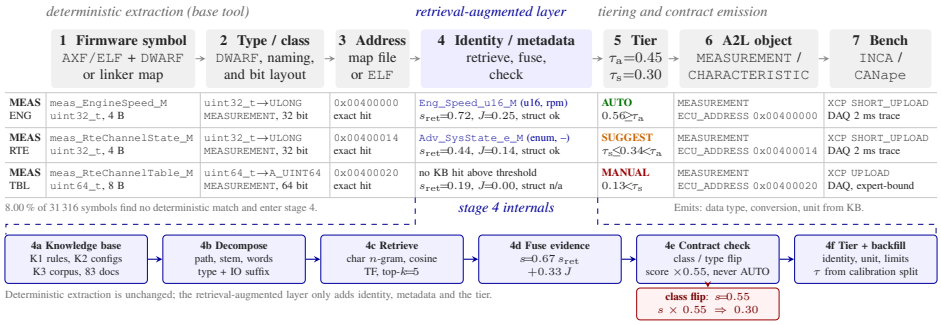



Fig. 2. Engineering walk-through on the released demonstration firmware. Columns 1–3 are the deterministic chain, column 4 the retrieval-augmented layer (stages 4a–4f), and columns 5–7 the tier (calibration-split thresholds), the A2L object, and the bench; in the red-boxed class-flip case the check multiplies the fused score by 0.55 (0.55 → 0.30), and the hard tier cap keeps a conflicting pair out of AUTO in any case.

stores behind the same interface; fusion and check are unchanged. Two evidence channels are fused per candidate:

$$s = w_r \cdot s_{\text{ret}} + w_t \cdot J(\tau_q, \tau_c), \tag{1}$$

where $s_{\text{ret}}$ is the retrieval score and $J$ the Jaccard overlap of decomposed tokens [20]. The weights $w_r, w_t$ are positive and sum to one; the shipped values $(2/3,\ 1/3)$ keep $s$ in $[0, 1]$ and make the two channels directly comparable. Exact hits short-circuit to $s{=}1.0$.

*Structural evidence: a check input, not a score term.* The I/O class and type family the naming convention carries are kept *out* of the fused score: the two fields fail independently, and an additive encoding lets one agreement mask the other ($+0.4$ for a class agreement cancels $-0.3$ for a type conflict), whereas a per-field rule does not.

*Post-fusion contract check.* Lexical and semantic evidence cannot distinguish a renamed variable from its class- or type-flipped twin: once the type and I/O suffixes are split off as structural signals, the remaining evidence is identical, and the mistake is a category error—a measurement object bound to a calibratable variable lands in the wrong section, and a wrong type width applies the wrong record layout. The check compares the two fields against the candidate: on a conflict it multiplies the fused score by 0.55 and *hard-caps* the tier at SUGGEST, so a conflicting alignment can never be auto-accepted; below $\tau_s$ it lands in MANUAL.

Each result is assigned a tier: **AUTO** ($s \geq \tau_a$; no human action), **SUGGEST** ($s \geq \tau_s$; one-click confirm), or **MANUAL**. Thresholds are *not* hand-picked: on a signal-disjoint calibration split we grid-search $\tau_a$ under valid-query precision $\geq 98\%$ and distractor false-auto $\leq 5\%$, and set $\tau_s$ to the largest value keeping AUTO∪SUGGEST precision $\geq 85\%$: a manual queue may absorb lower precision, an auto-accept may not. Each run ends with an *alignment report* (candidate, fused score, per-channel evidence, check flags) and a review queue; Fig. 2 traces the transformation end to end.

The same evidence completes missing metadata (units, descriptions). The consistency checker compares the signal set against existing A2L names on four axes: duplicates and type drift are deterministic (against the knowledge base's historical type), absent objects are flagged as orphans, and *rename recognition* reports a rename pair, not a removal plus addition.

### D. Implementation

A2L-RAGAlign extends an operational PyQt5-based A2L update tool (∼7,000 lines) with a self-contained prototype module (∼1,100 lines) and a thin adapter (∼160 lines); it passes 360 unit tests and runs on the standard library alone (optional dense retrievers excepted). A batch entry point slots into the address-matching stage, so the semantic layer sees exactly the symbols the deterministic chain missed, both behind an opt-in flag. The retrieval stage never mutates the regenerated description: candidates are reported alongside the deterministic ones, and the consistency check compares written objects against the existing header.

The tool, its embedded knowledge base, and the experiment harness are released as open source at https://github.com/jack0430-dev/A2L-RAGAlign_2026.

## IV. Experiments

### A. Setup

**Corpus and queries.** 83 signals spanning six functional groups (engine, battery, transmission, vehicle control, HVAC, advanced) of a transmission-control basic-software set come from the base tool's built-in demonstration dataset, so the benchmark needs no binary artifacts. Each signal goes through up to nine renaming transformations (lower-casing, camel-case, suffix drop, abbreviation and expansion of domain words, struct-path qualification, I/O token substitution, and two *multi-hop* combinations), yielding 631 valid queries; 120 further plausible non-existent signals (distractors) must not be auto-accepted, and 127 class- or type-flipped twins are demoted to human confirmation.

**Splits and calibration.** Signals and distractors are split 50/50 into calibration and test sets; confusables are evaluation-only. The test split holds 314 queries, 84 of them multi-hop—the hard column of Table II. Thresholds are calibrated on the calibration split ($\tau_a{=}0.45$, $\tau_s{=}0.30$); reported metrics are on the test split (with 60 distractors); the shipped defaults are more conservative ($\tau_a{=}0.85$, $\tau_s{=}0.50$). Baselines: exact

TABLE II
TEST-SPLIT ACCURACY (N=314 QUERIES, 83 SIGNALS; HARD = MULTI-HOP, N=84); THE CHECK GATES TIERING. RETRIEVAL-ONLY AND OURS SHARE THE IDENTICAL RANKING (THE CHECK CHANGES THE TIER, NOT THE ORDER) AND DIFFER IN TABLE III.

| Method | Top-1 (%) | Top-5 (%) | MRR | Hard Top-1 |
|---|---|---|---|---|
| Exact | 0.0 | 0.0 | 0.000 | – |
| Levenshtein | 92.4 | 98.1 | 0.946 | 78.6 |
| Token-Jaccard | 64.3 | 74.8 | 0.675 | 33.3 |
| BM25 (KB docs) | 96.8 | 100.0 | 0.980 | 95.2 |
| Retrieval-only | 99.0 | 100.0 | 0.994 | 97.6 |
| **Ours** | **99.0** | **100.0** | **0.994** | **97.6** |

TABLE III
SAFETY AT AUTO THRESHOLD 0.45: FALSE AUTO-ACCEPTANCE ON 127 CONFUSABLE PAIRS AND 60 DISTRACTORS.

| Configuration | Valid cov. | Conf. FP | Distr. FP |
|---|---|---|---|
| w/o type/IO penalty | 93.9 | 100.0 | 33.3 |
| Ours + struct. score term | 90.1 | 0.0 | 3.3 |
| Retrieval-only | 99.7 | 100.0 | 56.7 |
| Levenshtein | 86.0 | 100.0 | 96.7 |
| **Ours (full fusion)** | **93.6** | **0.0** | **1.7** |

matching, normalized Levenshtein [21] over raw names, token-Jaccard over our decomposition, BM25 [22] over the same documents, and the $n$-gram retriever alone, all with $k$=5. Control: all non-trivial baselines hold 100% Top-1 on the 83 unrenamed signals. Metrics are Top-1/Top-5 and MRR; Table III reports AUTO coverage on valid queries (Valid cov.) against false-acceptance rates on confusable pairs (Conf. FP) and distractors (Distr. FP).

### B. Alignment Accuracy on the Firmware-Evolution Workload

Table II shows the main comparison: exact matching scores 0% by construction, and Levenshtein degrades on multi-hop renames, where expansion erases character overlap. Fusion matches the best retrieval and adds the contract check, holding 100% Top-1 on seven of the nine transformations, where the strongest string baseline reaches 78.6% on the multi-hop subset. The three residual errors fall in the two abbreviation-heavy transformations (1 of 7 in *abbrev*, 2 of 42 in *camel_abbrev*), where the method only ties the baseline.

### C. Safety against Contract Violations

Ranking accuracy alone cannot distinguish a safe aligner from an unsafe one. Table III therefore evaluates the 127 confusable pairs, where auto-acceptance is by definition false, against auto coverage. The central evidence is that the post-fusion check, not the fused score, makes alignment safe: *every* configuration that drops it—pure retrieval, string similarity, and fusion without the type/IO penalty—blindly auto-accepts the flipped twins, whereas the full model eliminates confusable false-acceptance while giving up 0.3 coverage points against the checkless variant and cutting the distractor false-positive rate from 33.3% to 1.7%. The two channels are not interchangeable, and an ablation must remove a channel rather than reweight it: ranking the whole knowledge base by decomposition alone drops Top-1 from 99.0% to 64.3% (multi-hop: 97.6% to 33.3%), whereas dropping the token channel leaves the ranking untouched—the retriever already orders candidates well. The counterfactual row *Ours + struct. score term* puts the structural evidence back into the score, costing 3.5 coverage points and 1.6 points of distractor false positives and removing no false acceptance—hence a check input, not a score term.

### D. Tiers, Downstream Tasks, and Robustness

With the calibrated thresholds, 93.6% of the test queries are auto-accepted at 99.3% precision, 6.1% require a single confirmation, 0.3% are fully manual, and 1 of the 60 held-out distractors would be auto-accepted. Unit completion transfers the correct unit for 97.8% of 186 queries; consistency checking recognizes 100% of 188 renames and owns the remaining drift classes of Table I at 100% recall, no false orphans. Over 10 random splits, re-calibrating each time, Top-1 is $99.5\pm0.4\%$, hard-subset Top-1 $98.6\pm0.9\%$, AUTO coverage $93.5\pm1.4\%$ at precision $99.3\pm0.3\%$, and confusable false-acceptance is *zero* in every split; only the distractor false-positive rate is noisy ($1.8\pm2.3\%$), and the single-split figures above are the default seed.

**Threats to validity.** The renaming transformations approximate, but do not exhaust, real-world divergence. The 83-signal corpus covers scalar signals only, leaving bitfield-packed objects and two-dimensional maps unvalidated. The production evaluation inherits the image's own coverage: 98.4 % of the unresolved names have no counterpart in the address file, and a looser hit is not an identity—no renaming method recovers them.

### E. Production-Image Evaluation

The 83-signal corpus is controlled but small, so we repeat the evaluation on the released production image: its 4,003 address-file symbols—identifier, I/O class, and width, all the image carries—form the knowledge base, and we apply the six renaming classes this codebase exhibits (case convention, camel-case, abbreviation and expansion, member path, multi-hop), 180 sampled queries per configuration. A2L-RAGAlign reaches 97.8% Top-1 and 99.4% Top-5 on the full knowledge base, against 90.6% for Levenshtein, 88.3% for BM25, 72.2% for token-Jaccard, and 0% for exact matching. The retrieval sweep answers what the small corpus cannot: decomposition alone costs 18.3 Top-1 points at 83 candidates, growing to 25.6 at 4,003—the deficit grows with the candidate space. Retrieval-augmented is thus an operational claim, invisible on the small corpus.

## V. CONCLUSION

We presented A2L-RAGAlign, which restores the embedded measurement interface by knowledge-grounded fusion under calibrated tiers and a post-fusion contract check: 99.0% Top-1 at 93.6% auto-acceptance (99.3% precision) on the held-out split, zero false auto-acceptance on the 127 flipped twins where retrieval and string similarity fail, and a retrieval channel worth 25.6 Top-1 points on the production image.

## REFERENCES


[1] ASAM e.V., "ASAM MCD-1 XCP, Version 1.5.0: Universal Measurement and Calibration Protocol," 2017, Version 1.5.0, released 2017-11-30. [Online]. Available: https://www.asam.net/standards/detail/mcd-1-xcp/

[2] ASAM e.V., "ASAM MCD-2MC (ASAP2), Version 1.7.1: Data Model for ECU Measurement and Calibration," 2018, Version 1.7.1, released 2018-01-30. [Online]. Available: https://www.asam.net/standards/detail/mcd-2-mc/

[3] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, "What's in a Name? A Study of Identifiers," in *Proc. 14th IEEE International Conference on Program Comprehension (ICPC)*, 2006, pp. 3–12.

[4] D. W. Binkley, M. Davis, D. J. Lawrie, and C. Morrell, "To CamelCase or Under_Score," in *Proc. IEEE International Conference on Program Comprehension (ICPC)*, 2009, pp. 158–167.

[5] International Organization for Standardization, "ISO 26262-1:2018: Road Vehicles – Functional Safety – Part 1: Vocabulary," ISO Standard, 2018, edition 2, published 2018-12.

[6] ASAM e.V., "ASAM MDF: Measurement Data Format, Specification of the Measurement Data Format Used by Calibration and Analysis Tools," 2025, Version 4.3.0, released 2025-09-23. [Online]. Available: https://www.asam.net/standards/detail/mdf/

[7] International Organization for Standardization, "ISO 14229-1:2026: Road Vehicles – Unified Diagnostic Services (UDS) – Part 1: Application Layer," ISO Standard, 2026, edition 4, published 2026-06; supersedes the withdrawn ISO 14229-1:2020.

[8] International Organization for Standardization, "ISO 11898-1:2024: Road Vehicles – Controller Area Network (CAN) – Part 1: Data Link Layer and Physical Coding Sublayer," ISO Standard, 2024, edition 3, published 2024-05; supersedes the withdrawn ISO 11898-1:2015.

[9] AUTOSAR Consortium, "AUTOSAR Technical Overview," 2018, Release R4.4.0, released 2018-10-31. [Online]. Available: https://www.autosar.org/standards/classic-platform/

[10] DWARF Committee, "DWARF Debugging Information Format, Version 5," 2017, approved 2017-02-13; DWARF 6 is still a draft. [Online]. Available: https://dwarfstd.org/

[11] E. Rahm and P. A. Bernstein, "A Survey of Approaches to Automatic Schema Matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[12] M. Bilenko, R. Mooney, W. Cohen, P. Ravikumar, and S. Fienberg, "Adaptive Name Matching in Information Integration," *IEEE Intelligent Systems*, vol. 18, no. 5, pp. 16–23, 2003.

[13] A. Fan *et al.*, "Large Language Models for Software Engineering: Survey and Open Problems," in *Proc. IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 2023, pp. 31–53.

[14] X. Hou *et al.*, "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 33, no. 8, pp. 1–79, 2024.

[15] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.

[16] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[17] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 6769–6781. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/

[18] X. Gu, H. Zhang, and S. Kim, "Deep Code Search," in *Proc. 40th ACM/IEEE International Conference on Software Engineering (ICSE)*, 2018, pp. 933–944.

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410/

[20] P. Jaccard, "Étude comparative de la distribution florale dans une portion des Alpes et du Jura," *Bulletin de la Société Vaudoise des Sciences Naturelles*, vol. 37, pp. 547–579, 1901.

[21] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[22] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.